

RULEBLIND-DCRT: Diagnosability-Conserving Repair Transactions for Stateful Agents

Rohini Arunachalam
Anil Neerukonda Institute of
Technology & Sciences
Visakhapatnam, India
arohini.cse@anits.edu.in

Gnanodhay Randhi
Anil Neerukonda Institute of
Technology & Sciences
Visakhapatnam, India
randhignanodhay@gmail.com

Hema Raju Barri
Johns Hopkins University
Baltimore, MD, USA
bhemaraju.138@gmail.com

Abstract—Destructive repair of a stateful agent can erase evidence needed for later diagnosis. We introduce diagnosability-conserving repair transactions (DCRT): before repair, a trusted gate executes an authorized measurement plan, durably stores a residual, and admits the write only if the target remains decodable from that residual and certified poststate. For a fixed plan, the minimum one-shot residual alphabet equals the chromatic number of an operational characteristic graph; authorized plan selection is weighted set cover, coupled to coloring under a bit cap. The compiler receives finite target tables, while callbacks receive neither target nor hypothesis label. Across a 1,480-world synthetic study and 22,200 semantic transactions, DCRT preserves exact targets; no retention after global reset achieves 1/37. Three constructed native microbenchmarks add 4,000 executions: DCRT, RAW-W, and native checkpoints recover 100%, versus 15% without retention; DCRT and RAW-W require two or three bits. A workload-constructed LangGraph/SQLite case adds 640 executions. DCRT and a phase-matched one-bit RAW-W control recover 160/160 future procedures; median ledger sizes are 1,829 and 1,475 bytes, respectively. In that case, DCRT retains less than an 8,536.5-byte full-U snapshot and 96,256-byte SQLite backup, but has higher median latency than all controls. All 32 DCRT hard-exit cases converge under idempotent recovery. In 100 local-service cells, plain DCRT preserves T but reaches one final non-idempotent mutation in only 6/8 durably admitted schedules; a receiver-side stable key reaches 8/8. These constructed local cases do not establish distributed atomicity.

Keywords—autonomous agents, fault diagnosis, trustworthy repair, transaction processing, observability, zero-error coding

I. INTRODUCTION

A repair can make an agent look healthy while destroying the only evidence that distinguishes why it failed. Once a memory is compacted, a deployment is reset, or a tool registry is normalized, different fault histories may map to the same live state. That loss matters when the next action depends on the original cause: revoke a credential, replay a cursor, reconcile a payment, or probe an inventory reservation. Full checkpointing preserves evidence but can retain irrelevant or privileged state; an ad hoc trigger log gives no target-specific sufficiency guarantee.

The difficult problem is therefore prospective: before an information-destroying write, and without already knowing the diagnosis, preserve a feasible *future diagnostic procedure*. Let a trusted gate acquire only authorized state-derived observations, encode a bounded residual, and refuse the repair unless that residual plus declared post-repair information recovers a chosen target. A graph coloring on hidden fault labels would be unusable because an online encoder cannot evaluate the unknown label. DCRT instead colors the measurement vectors the gate can actually obtain.

This paper contributes a target-relative conservation contract, a certificate-producing privilege-aware compiler, and a write-ahead runtime. For a fixed measurement plan we

prove the operational characteristic-graph bound; without a payload cap, minimum acquisition is weighted set cover, while a cap requires joint plan/code search. The evaluated bounded instances are exact; unresolved bounds fail closed as diagnostic debt. The runtime rejects unsupported outcomes, fsyncs a compact code before effect, checks trusted postconditions, and carries one declared target through multiple admitted repairs. We also isolate a complementary limit: local diagnostic durability cannot by itself make an unkeyed external effect exactly once.

Evaluation combines an exhaustive finite benchmark, three constructed native-persistence microbenchmarks, a framework-authentic but workload-constructed LangGraph case, and process-crash injection. The experiments include matched repair/postconditions, a four-phase raw-measurement transaction, reconstructive checkpoint baselines, storage and warm online latency, privilege ablations, and external effects. They also expose unfavorable outcomes: DCRT has higher median latency in every measured system and is larger than checkpoints for two small native artifacts. The contribution is certified target sufficiency and admission, not universal performance superiority.

II. RELATED WORK AND GAP

Diagnosability asks whether observations distinguish fault behaviors [1]. Diagnosis with repair dates at least to Russell and Kime [2]. Safe diagnosability requires fault detection before unsafe behavior, while active fault-tolerant control uses online diagnosis to switch or reconfigure control policies [3], [4]. Sensor-activation and diagnosability-planning methods select observations or actions to establish or preserve diagnosability [5], [6]. DCRT does not claim these ideas. Its object is a gate on an already proposed destructive transition: acquire and durably preserve just enough authorized information before admitting that transition.

Witsenhausen characterized zero-error coding with decoder side information through graph coloring [7], while minimum test cover and sensor placement select observations that separate required pairs [8]. Koulgi *et al.* study broader zero-error codes with side information [9]. DCRT instantiates these foundations at two distinct layers. Set cover charges observation cost and privilege; coloring bounds the message *after an executable observation is acquired*. Coloring raw hypotheses is only an oracle lower bound unless the color factors through a live measurement.

Recent agent systems already provide transactions, repair, and evidence. Mnemosyne validates and repairs workflows and retains repair-trigger evidence [10]; Cordon stages irreversible tool effects with recovery records [11]; MemTX validates belief commits and cascading repairs [12]. EvidenT preserves histories and artifacts for iterative package repair [13]. Failure Sketches and Gleaner illustrate how diagnosis-oriented representations and selective retention can reduce

trace volume [14], [15]. Execution-state unlearning uses checkpointing and replay for the nearly dual goal of removing historical influence [16], and AttriGuard gates tool calls through attribution and shadow replay [17]. Classical adaptive state-identification experiments distinguish current labeled-transition-system states through input/output tests [18]. LangGraph supplies thread-scoped checkpoint persistence rather than a diagnostic conservation contract [19]. Write-ahead recovery is classical [20], while idempotent message-oriented design addresses non-atomic cross-service effects [21]; Proposition 3 specializes that familiar boundary to DCRT evidence, not a new exactly-once protocol. Among the systems examined, none couples a repair-induced graph over executable measurement vectors to privilege-constrained residual synthesis and fail-closed precommit enforcement. This is a scoped search finding, not a priority claim.

III. DCRT MODEL

A. Operational conservation

Let the finite modeled world be θ in Θ and the declared target be $T=\tau(\theta)$. A target may be an original fault set or the identity of a future diagnostic procedure. Let U be the complete transcript of the declared pre-repair procedure; first require T to be a function of U , or diagnosis was not feasible before repair. A proposed repair r leaves post information $V=v_r(\theta)$, including realized state, repair identity, and prior ledger entries. Before repair the gate executes one hypothesis-independent authorized bundle and obtains $W=w(\theta)$. It stores $Z=e(W)$, never $e(\theta)$ or $e(T)$, and requires a decoder d such that

$$d(Z, V) = T \quad \text{for every } \theta \text{ in } \Theta. \quad (1)$$

For a fixed executable measurement bundle S , define $\Gamma_{r,s}$ on attainable W values. Distinct w and w' are adjacent when some corresponding worlds require different targets yet share a post observation:

$$\begin{aligned} \{w, w'\} \text{ in } E(\Gamma_{r,s}) \text{ iff exists } \theta, \theta' : \\ W_\theta = w, \quad W_{\theta'} = w', \\ T_\theta \neq T_{\theta'}, \quad V_\theta = V_{\theta'}. \end{aligned} \quad (2)$$

Theorem 1 (operational residual). For fixed S such that T is a function of (W, V) , the minimum alphabet of a deterministic one-shot residual satisfying (1) is $\chi(\Gamma_{r,s})$; otherwise no such residual exists. The minimum worst-case fixed-length payload is the ceiling of the base-two logarithm of that alphabet size.

Proof. Adjacent measurements cannot share a code because their common V would require two decoder outputs, so every valid encoder is a proper coloring. Conversely, feasibility excludes conflicting targets at identical (W, V) , and a proper coloring excludes them at distinct equally colored W . Every supported (color, V) therefore determines at most one target. Returning it defines a decoder. The statement excludes variable-length, block, noisy, and randomized codes.

This operational graph matters. Six worlds can produce three W values A, B, C whose post fibers confound $A-B, B-C$, and $C-A$. The raw-hypothesis graph can be three disjoint edges and two-colorable, while the graph on executable values is a triangle and needs three symbols. A hidden-label coloring would smuggle an unavailable diagnosis into the encoder.

B. Authorized measurement synthesis

Suppose deterministic jointly executable measurements m_j have acquisition cost c_j and required privilege set R_j . For granted privileges G , let M_G collect the j whose R_j are contained in G ; every plan below uses a subset S of M_G . Define C_r as the pairs $\{\theta, \theta'\}$ for which $\tau(\theta) \neq \tau(\theta')$ and $V(\theta) = V(\theta')$. A measurement covers a pair when its outcomes differ. Without a payload cap, a minimum-cost static plan is weighted set cover:

$$\begin{aligned} S^* = \arg \min_S \sum_{j \text{ in } S} c_j \\ \text{s.t. every } p \text{ in } C_r \text{ is covered.} \end{aligned} \quad (3)$$

With a B -bit cap, acquisition and coding must be solved jointly:

$$\begin{aligned} S_B^* = \arg \min_S \sum_{j \text{ in } S} c_j \\ \text{s.t. } S \text{ covers } C_r \text{ and } \chi(\Gamma_{r,s}) \leq 2^B. \end{aligned} \quad (4)$$

A richer W can cost more yet require fewer stored colors. The finite compiler uses exact bit-mask dynamic programming for unconstrained set cover and bounded subset enumeration with certified coloring bounds under a cap. Evaluated instances are exact; unresolved bounds fail closed. It returns **exact**, **debt**, or **impossible** with collapse witnesses. Debt denotes a privilege, budget, or certification obstruction. Impossible means either T was not recoverable from U or the declared catalog cannot separate a required pair. Certificates bind canonical pre/post/target tables, measurement definitions, privileges, repair IDs, budgets, codebook, decoder, and coloring bounds.

C. Runtime, composition, and external effects

The runtime executes OBSERVE, PREPARE, APPLY, VERIFY, and COMMIT after offline profiling and compilation. It resolves a content-addressed certificate, executes only selected state-derived measurements, and rejects vectors outside compiled support. PREPARE appends the code, repair list, and artifact digests to a hash-chained JSONL ledger, then flushes and fsyncs before any repair callback. APPLY gives callbacks runtime state handles, never hidden labels; a callback can still have an external effect. VERIFY checks trusted postconditions and a supported (Z, V) decoder entry. Failure prevents COMMITTED but does not undo an already executed effect.

Let Y_t contain live observations, accumulated durable residuals, repair identities, and realized outcomes after t transactions. Write $P \leq Q$ when partition P refines Q . Admission requires a decoder for the same declared T after each step, hence

$$\Pi(Y_t) \leq \Pi(T). \quad (5)$$

Lemma 2 (sequential conservation). If T is recoverable from Y_0 and every step is admitted against T , then T remains recoverable from Y_t at every depth. *Proof.* Induct using the decoder certified at each step. The premise fixes the original universe and target and requires durable residuals, accurate transitions, and all adaptive branches.

Proposition 3 (external-effect indistinguishability).

Assume a non-idempotent external effect shares no atomic commit with local durable state, exposes neither a queryable outcome nor a stable deduplication key, and recovery observes only local evidence L . There exist a history h_0 that crashes immediately before the effect and h_1 in which the effect commits before a crash preceding local acknowledgment, with $L(h_0)=L(h_1)$. No deterministic local recovery guarantees one effect in both. *Proof.* Equal L forces the same retry/skip decision: retry duplicates h_1 ; skip misses h_0 . Randomization cannot provide zero-error completion. DCRT preserves T , not effect occurrence. Receiver-side atomic deduplication under a stable key makes retry safe; an outbox plus such a key can preserve delivery under eventual successful retry, but without a diagnostic residual need not preserve T .

The certificate binds declared IDs and finite model tables, not callback binaries. Same-ID implementation substitution is not detected and violates the trusted-binding assumption. Likewise, an unkeyed local hash chain detects accidental corruption, not an adversary able to recompute or roll back it.

IV. IMPLEMENTATION AND EVALUATION

A. Executable artifact

The Python implementation provides `compile_dcrt`, immutable certificates, certified color-code and exact-decode paths, `RuntimeRepair`, a content-addressed registry, and an fsyncing write-ahead ledger. Runtime callbacks receive state handles rather than world identifiers or benchmark targets. The accompanying reproducibility package contains source, frozen protocols, raw rows, content hashes, and independent verifiers; one rebuilds certificates from model sidecars. A post-development oracle exhaustively checked every measurement subset, coloring, and decoder row for 5,000 random small models and 30,000 compiler calls; all decisions matched. The result verifier recomputes denominators, medians, nearest-rank p95 values, file hashes, and crash outcomes from those rows.

B. Protocols and baselines

The synthetic study fixes 40 eight-component administrative-agent records. Each contains healthy, eight singleton, and 28 double-fault worlds (37/record; 1,480 total), with the exact original fault set as T . Five fixed histories give 15 transactions/world and 22,200 total. The records reuse five query texts and 15 structural certificate contexts; they are not 40 independent system designs. These semantic transactions use an in-memory evaluation ledger. One representative transaction and focused tests exercise JSONL PREPARED-fsync-before-repair ordering; the native/framework trials below are file-backed. A 2,240-world triple-fault set is held outside the main certificate. Protocols and later addenda are development-informed, not blind preregistrations.

Three constructed native-persistence microbenchmarks exercise actual APIs: (i) SQLite conversational memory with stale-summary and conflicting-fact violations; (ii) a filesystem deployment with model, prompt, and route drift; and (iii) a JSON permission registry with missing, excess-network, and executable-handler-digest faults. Their complete powersets contain 4, 8, and 8 worlds. Repairs delete/rebuild and reopen native artifacts. Each strategy starts from a byte-matched

fixture and performs the same repair and postcondition. No retention is a lower bound; a full native checkpoint is fsynced and reconstructive; RAW-W and DCRT run identical probes and four fsynced phases. Five warmups and 50 measured repetitions per world/method yield 400 warmups and 4,000 warm online executions. The order rotates deterministically. Compilation and cold installation are excluded; DCRT includes warm artifact validation.

The framework-authentic but workload-constructed deterministic LangGraph case pins `langgraph 1.2.11` and `langgraph-checkpoint-sqlite 3.1.1` [19]. Eight authored `StateGraph+SqliteSaver` histories cross two surviving domains, two erased error branches, and attempt depths one/three. T is one of four authored future procedures. W is only the error-branch bit: each value remains compatible with one calendar and one commerce target. V retains domain, so (W,V) is exact without pre-repair diagnosis. Attempt depth and 4,096 transcript bytes/attempt are target-irrelevant and deliberately vary full-state retention. DCRT, a phase-matched one-bit RAW-W transaction, a fsynced full-U JSON snapshot, and SQLite's backup API perform the same compaction, idempotent application write, target recovery, and diagnostic probe. RAW-W uses a hand-coded $W+V$ decoder but no certificate or admission check. Three warmups and 20 measured trials/world/method give 640 timed executions. No retention chooses one first procedure from V before any probe and is scored once on all eight worlds.

Crash evaluation has two parts. The LangGraph DCRT adapter injects `os._exit(86)` at four durable boundaries for every world (32 children) and runs idempotent recovery; the three controls are not crash-tested there. A separate exhaustive state-machine matrix crosses five strategies, idempotent/non-idempotent local-service effects, and ten schedules: 100 children, 90 hard exits. Strategies are no retention, raw checkpoint, DCRT, DCRT plus a stable receiver key, and an outbox plus key deliberately carrying no diagnostic residual. All results are deterministic coverage, not statistical samples.

V. RESULTS

A. Exact conservation and falsification

All 1,480 main starts already have the correct majority decision, so the main study measures latent-fault conservation rather than repair efficacy. DCRT commits and later decodes the exact original target in all 22,200 transactions. Of their callbacks, 11,640 change no component state and 10,560 touch a currently faulty component; this is exhaustive coverage, not independent replication. A global reset makes every poststate identical: no-retention accuracy is $1/37=.027$ with worst ambiguity 37, while DCRT needs six bits for the 37-clique. Four staged histories preserve the target with eight incremental bits; this append-only total is locally step-optimal, not globally sequence-optimal. Fig. 1 shows the protocol and no-retention loss by depth.

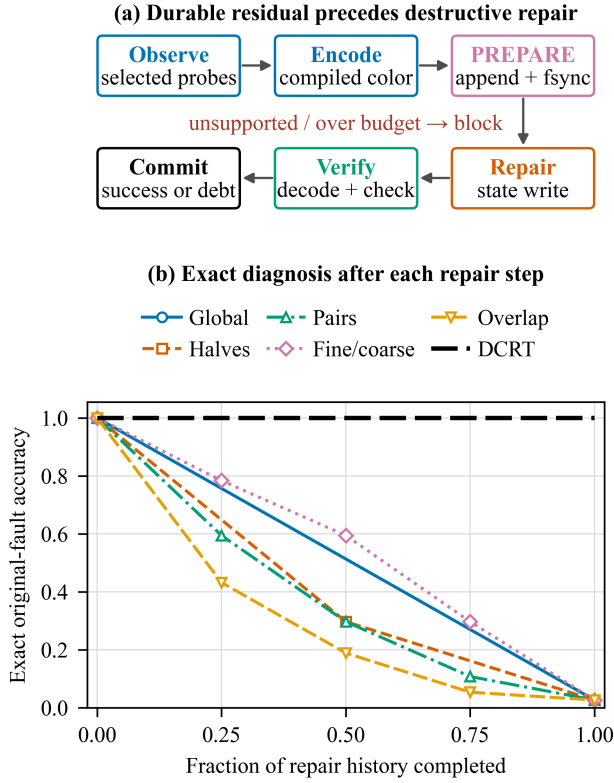


Fig. 1. (a) DCRT encodes an executable authorized residual before destructive repair. (b) Across five fixed histories, no-retention exact diagnosis declines with erased state while composed DCRT remains 1.0.

Required negative controls fail closed. A five-bit global cap and cost budget seven are below certified minima; removing any singleton privilege leaves a collapse witness. All 2,240 tested triple-fault vectors lie outside global-certificate support and are rejected before PREPARED, although an unknown vector that aliases support need not be detected. An undeclared extra write yields debt and no commit, but its physical mutation remains. A separate 93-world certificate covers healthy, one-, two-, and five-fault states: it corrects all 2,240 initially wrong five-fault decisions and decodes all 3,720 targets with seven bits. A six-world router witness shows the joint tradeoff: a cost-one probe needs two bits, whereas a one-bit cap selects a cost-two probe.

B. Native and framework repairs

Table I reports exact target recovery; each preserving method is 1.000. Column two gives per-preserving-method trials/no-retention worlds; the last column gives DCRT residual bits, acquisition-cost units, and pre-repair W read scopes. Trusted V access is outside the optimizer. The native adapters begin wrong or unsafe in 850/1,000 measured executions per method and end correct in 1,000/1,000, removing 1,400 violation instances. Their 2,800 DCRT information bits equal RAW-W, so there is no compression advantage. All eight one-at-a-time compile-time scope ablations return debt. In one live revocation per adapter, the adapter writes only FAILED: zero measurements, PREPARED records, or callbacks, and unchanged native state. This enforcement is in the adapter, not the reusable core. LangGraph's no-scope compilation returns debt; its live revocation was not tested.

TABLE I. TARGET RECOVERY AND REQUIRED SCOPES

Workload	trials P/N	No ret.	Pres.	bits/cost/scopes
SQLite memory	200/4	0.250	1.000	2/2/2
Filesystem deploy	400/8	0.125	1.000	3/3/3
Tool registry	400/8	0.125	1.000	3/3/3
LangGraph SQLite	160/8	0.500	1.000	1/1/1

Table II gives median warm online milliseconds and retained bytes per trial; kB is decimal. D/W/U/C denote DCRT, phase-matched RAW-W, full-U JSON snapshot, and native checkpoint. DCRT's shared artifacts and the shared RAW-W decoder code are excluded here; DCRT artifact sizes are reported below. All native D/W byte values happen to be constant; checkpoint values shown are medians. Checkpoints use the same repair/postcondition and a durable reconstructive copy, but no fabricated four-phase envelope. Formats are not serialization-equivalent and values are comparable only within rows. Native runs use Python 3.13.2/SQLite 3.45.3; LangGraph uses isolated Python 3.14.7/SQLite 3.53.4 on the same M3/16-GB host. Every DCRT and RAW-W transaction starts with an empty ledger.

TABLE II. MEDIAN WARM LATENCY AND PHYSICAL RETENTION (D/W/U/C)

Workload	Latency D/W/U/C (ms)	Bytes D/W/U/C
SQLite memory	2.645/2.099/—/1.577	1.97k/1.76k/—/20.48k
Filesystem deploy	2.470/1.598/—/1.523	1.99k/1.80k/—/0.399k
Tool registry	133.696/131.671/—/127.242	2.01k/1.82k/—/0.737k
LangGraph SQLite	6.863/5.193/4.534/4.994	1.83k/1.48k/8.54k/96.26k

Nearest-rank D/W/C p95 values are 4.256/3.317/2.832 ms for memory, 3.005/2.109/1.963 for deployment, and 145.471/141.952/136.271 for the registry; LangGraph D/W/U/C is 9.361/7.610/6.890/7.906 ms. Every raw sample is in the artifact. DCRT has higher median latency than every comparator. It is about 11–12% larger than native RAW-W and 24.0% larger than LangGraph RAW-W, isolating the price of certification and admission metadata. It is also 4.99×/2.73× the tiny deployment/registry checkpoints, but 90.4% smaller than the memory checkpoint. Against LangGraph full-state retention, DCRT is 78.6% smaller than raw U and 98.1% smaller than backup. Shared LangGraph certificate/recovery artifacts add 75,589/882 bytes; native pairs range from 6,866/792 to 29,999/1,394 bytes. At median sizes the LangGraph shared bytes amortize against full-U snapshots once 12 transactions share them, but never against RAW-W because DCRT's marginal ledger is larger. DCRT adds automatic authorized synthesis, a sufficiency certificate, and fail-closed enforcement.

In LangGraph, DCRT, RAW-W, raw snapshot, and backup each recover the target and execute the confirmed future probe in 160/160 trials. No retention selects the exact first procedure before one probe in 4/8 worlds; an absent outcome is not reused for a second call. Allowing adaptive probes would change the permitted post-repair

observation/evaluation contract and is not evaluated. The one-bit code alone is deliberately ambiguous over two targets, so DCRT success genuinely uses V rather than storing a diagnosed procedure. All 32 DCRT hard-exit cases reopen the ledger and both databases, converge to one idempotent effect row and normalized checkpoint, recover T, execute the confirmed probe, and end COMMITTED. Effect convergence here depends on the idempotent application write and recovery, not DCRT alone.

C. External-effect boundary

The separate crash matrix covers $5 \times 2 \times 10 = 100$ unique cells; 90 terminate through `os._exit(86)`. Table III conditions on the eight durably admitted schedules for one fixed request in the non-idempotent mode; pre-intent exits are excluded. Raw checkpoint and plain DCRT recover T in 8/8 but conservatively leave two effects unperformed and flag three schedules for manual resolution. Manual is an overlapping recovery-required flag, not a mutually exclusive outcome. DCRT plus receiver-side atomic key deduplication reaches one final business mutation in 8/8. The keyed outbox also reaches 8/8 but recovers 0/8 targets because its residual was deliberately omitted. For the idempotent-set mode, all four admitted retention/outbox strategies reach one final mutation in 8/8, although a replay can make two service attempts.

TABLE III. NON-IDEMPOTENT LOCAL-EFFECT CRASH OUTCOMES

Strategy	T exact	One mutation	Miss	Manual
Raw checkpoint	8/8	6/8	2/8	3/8
DCRT	8/8	6/8	2/8	3/8
DCRT + receiver key	8/8	8/8	0/8	0/8
Outbox + key, no Z	0/8	8/8	0/8	0/8

The matrix realizes Proposition 3: crashing before versus after the service mutation leaves byte-identical declared recovery inputs (native state, PREPARED ledger, and registry) but service counts zero versus one. Recovery is not passed the schedule and does not query evaluator-only service truth. Harness configuration and service databases differ, so we do not claim all persisted files are identical. The stable-key result belongs to receiver-side deduplication, not DCRT alone.

VI. DISCUSSION AND LIMITATIONS

DCRT solves a narrow question: can this gate, with this finite model, target, measurement catalog, privilege set, future observer, and proposed repair, preserve a feasible diagnosis? It does so without an online target label. It does not reconstruct all prestate, make a target learnable when U was insufficient, or guarantee that an unsupported world will be recognized. The trusted healthy references, measurement semantics, repair transition, verifier, post observer, filesystem, and registry are assumptions. Exact planning is exponential in catalog/graph size and is demonstrated only at these scales.

The three native systems are constructed consistency tasks with direct probes, not difficult representation learning, naturalistic fault prevalence, or independent workloads. Repetitions characterize one machine; they support no confidence intervals or latency generalization. Timing

excludes fixtures, model construction, compilation, and cold install. Checkpoints and raw snapshots preserve information beyond T and support forensic questions DCRT cannot answer; byte comparisons cross JSON, JSONL, directory snapshots, and SQLite allocation. Registry costs are amortizable only if artifacts are safely shared. The tool-registry runtime is dominated by launching an executable handler.

Crash tests terminate a local process at semantic boundaries against SQLite files. They do not cover power loss, torn writes, a remote API, network partitions, concurrency, Byzantine storage, malicious callbacks, or arbitrary interleavings. One local service file is separate from the transaction ledger but runs in the same child. Exactly-one keyed outcomes depend on emulator-side atomic deduplication. Outbox target failure is by construction and does not imply that outboxes cannot carry diagnostic residuals. Production use needs attested callback/capability bindings, authenticated ledgers and trusted heads, real effect APIs, compensation or shadow-state commit, concurrent-writer control, and evolving/unknown-world policy. Noisy, adaptive, or stochastic measurements require a different certificate.

VII. CONCLUSION

DCRT makes future diagnosability an admission condition instead of an accidental by-product of logging. It synthesizes an executable authorized observation, stores only a certified repair-induced residual before effect, and composes the declared target across destructive writes. Exact synthetic, native-persistence, and LangGraph executions show that diagnosis can survive while live evidence is erased; the measurements also show nontrivial latency and metadata costs. Crash injection draws the boundary sharply: DCRT can preserve *what to diagnose*, while receiver-side atomic deduplication under a stable key or another effect protocol is separately required to complete non-idempotent work safely.

REFERENCES

- [1] M. Sampath, R. Sengupta, S. Lafortune, K. Sinnamohideen, and D. Teneketzis, "Diagnosability of discrete-event systems," *IEEE Trans. Autom. Control*, vol. 40, no. 9, pp. 1555-1575, 1995, doi: 10.1109/9.412626.
- [2] J. D. Russell and C. R. Kime, "System fault diagnosis: Closure and diagnosability with repair," *IEEE Trans. Comput.*, vol. C-24, no. 11, pp. 1078-1089, 1975, doi: 10.1109/T-C.1975.224136.
- [3] A. Paoli and S. Lafortune, "Safe diagnosability for fault-tolerant supervision of discrete-event systems," *Automatica*, vol. 41, no. 8, pp. 1335-1347, 2005, doi: 10.1016/j.automatica.2005.03.017.
- [4] A. Paoli, M. Sartini, and S. Lafortune, "Active fault tolerant control of discrete event systems using online diagnostics," *Automatica*, vol. 47, no. 4, pp. 639-649, 2011, doi: 10.1016/j.automatica.2011.01.007.
- [5] W. Wang, S. Lafortune, A. R. Girard, and F. Lin, "Optimal sensor activation for diagnosing discrete event systems," *Automatica*, vol. 46, no. 7, pp. 1165-1175, 2010, doi: 10.1016/j.automatica.2010.04.004.
- [6] H. Ibrahim, P. Dague, A. Grastien, L. Ye, and L. Simon, "Diagnosability planning for controllable discrete event systems," in *Proc. 31st AAAI Conf. Artif. Intell. (AAAI-17)*, vol. 31, no. 1, pp. 1149-1155, 2017, doi: 10.1609/aaai.v31i1.10694.
- [7] H. S. Witsenhausen, "The zero-error side information problem and chromatic numbers (Corresp.)," *IEEE Trans. Inf. Theory*, vol. 22, no. 5, pp. 592-593, 1976, doi: 10.1109/TIT.1976.1055607.
- [8] L. Sela Perelman, W. Abbas, X. Koutsoukos, and S. Amin, "Sensor placement for fault location identification in water networks: A minimum test cover approach," *Automatica*, vol. 72, pp. 166-176, 2016, doi: 10.1016/j.automatica.2016.06.005.

- [9] P. Koulgi, E. Tuncel, S. L. Regunathan, and K. Rose, "On zero-error source coding with decoder side information," *IEEE Trans. Inf. Theory*, vol. 49, no. 1, pp. 99-111, 2003, doi: 10.1109/TIT.2002.806154.
- [10] E. Y. Chang and L. Geng, "Mnemosyne: Agentic transaction processing for validating and repairing AI-generated workflows," arXiv:2607.00269, 2026.
- [11] Z. Chen, H. Liu, D. Xu, D. Dong, J. Li, B. Pu, and J. Zhai, "Cordon: Semantic transactions for tool-using LLM agents," arXiv:2606.17573, 2026.
- [12] X. Li *et al.*, "MemTX: Transactional belief commit for stateful agent memory," arXiv:2607.23929, 2026.
- [13] C. Zhao *et al.*, "EvidenT: An evidence-preserving framework for iterative system-level package repair," to appear in *Proc. ACM Softw. Eng. (ISSTA)*, 2026, arXiv:2605.08621.
- [14] B. Kasikci, C. Pereira, G. Pokam, B. Schubert, M. Musuvathi, and G. Candea, "Failure sketches: A better way to debug," in *Proc. HotOS XV*, 2015.
- [15] Y. Yang, A. Fang, S. Zhang, and P. He, "Gleaner: A semantically-rich and efficient online sampler for microservice diagnostics," arXiv:2604.16810, 2026, accepted at ISSTA 2026.
- [16] C. Yao *et al.*, "Forgetting without restarting: Execution-state unlearning for stateful LLM agents," arXiv:2609.04875, 2026.
- [17] Y. He, H. Zhu, Y. Li, S. Shao, H. Yao, Z. Liu, and Z. Qin, "AttriGuard: Defeating indirect prompt injection in LLM agents via causal attribution of tool invocations," in *Proc. USENIX Security*, pp. 1547-1566, 2026.
- [18] P. van den Bos and F. Vaandrager, "State identification for labeled transition systems with inputs and outputs," *Sci. Comput. Program.*, vol. 209, Art. no. 102678, 2021, doi: 10.1016/j.scico.2021.102678.
- [19] LangChain, "LangGraph persistence and releases 1.2.11/3.1.1," 2026. [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/persistence>; <https://github.com/langchain-ai/langgraph/releases/tag/1.2.11>; <https://github.com/langchain-ai/langgraph/releases/tag/checkpointsqlite%3D%3D3.1.1>. Accessed: Sep. 10, 2026.
- [20] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.
- [21] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proc. CIDR*, pp. 132-141, 2007.